SCOPPHU · AI ENGINEERING COURSE 2026 · MÓDULO 02

Data Pipelines com Apache Airflow

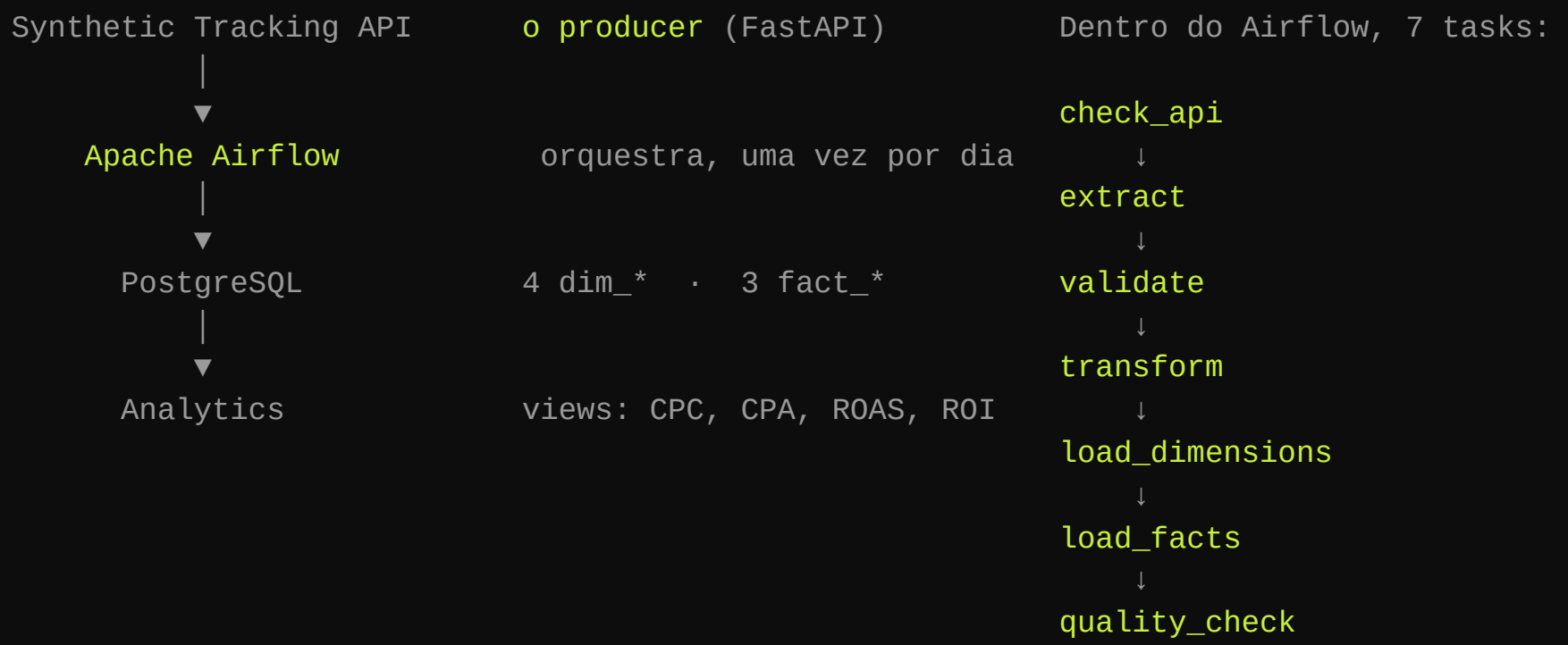
Dos dados à operação — e o que acontece quando tudo começa a falhar

Synthetic Tracking API → Apache Airflow → PostgreSQL

Apolonia Group · 5 horas · Python · Pandas · FastAPI · Pydantic · Airflow · PostgreSQL · Docker

ABERTURA

Hoje vamos construir isto



Três containers: `api`, `airflow`, `postgres`. E **duas bases de dados** no mesmo Postgres: `analytics` (o negócio) e `airflow` (os metadados do orquestrador). Confundi-las é o erro mais comum da primeira semana.

ABERTURA

O objetivo não é aprender Airflow

O objetivo é perceber como construir uma **pipeline de dados fiável**.

Python | Pandas | FastAPI | Pydantic | **Airflow** | PostgreSQL | Docker

O Airflow é uma das sete peças. É a que organiza as outras — e é a última que vamos montar.

ABERTURA · O PROBLEMA

O problema

O QUE O NEGÓCIO GERA

- **custo** — tráfego comprado, cobrado *à hora*
- **clicks e impressions** — também agregados por hora
- **conversões** — postbacks, uma a uma, com receita
- **daily control** — o agregado diário, vindo de *outro* sistema

O QUE O NEGÓCIO PERGUNTA

- Quanto gastámos ontem?
- Quanto ganhámos ontem?
- Que campanhas dão lucro?

E, sobretudo: **todos os dias, antes das 9h.**

Os dados nascem em dois sistemas diferentes. Alguém tem de os ir buscar, pô-los de acordo, e deixá-los onde se possam somar.

ABERTURA · O PROBLEMA

O difícil não é mover dados

API ——————→ PostgreSQL cinquenta linhas de Python

API em baixo · timeout · HTTP 429 · schema drift · valores impossíveis · a mesma run outra vez · dois sistemas a discordar

Mover dados é fácil.
Fazê-lo de forma fiável é o trabalho.

O DOMÍNIO E O MODELO

Como é que isto funciona, no mundo real

1 · COMPRA-SE TRÁFEGO

Uma **campanha**, de um **workspace**, compra tráfego numa **fonte** (ExoClick, TrafficStars…).

O custo, os clicks e as impressions chegam **agregados por hora** — e por *investment provider*.

2 · ALGUÉM CONVERTE

A rede de afiliação envia um **postback**: é uma **conversão** de uma **oferta**, com a sua `revenue`.

Chega **evento a evento**, e sempre *depois* da visita que a originou.

3 · O BACKOFFICE FECHA O DIA

Um segundo sistema mantém o **daily control**: receita e investimento por campanha e por dia.

Às vezes **escrito à mão** (`is_manual`). É uma segunda opinião sobre os mesmos números.

Na realidade isto vive em dois sítios: Mongo (eventos) e Postgres (gestão e daily control). A nossa API expõe os dois lados num só serviço.

O DOMÍNIO E O MODELO

Antes do Airflow: o que representa uma linha?

FACT_CONVERSIONS
1 linha = 1 conversão
um postback

FACT_CAMPAIGN_COSTS
1 campanha × 1 hora × 1 provider
o custo nunca é por evento

FACT_DAILY_CONTROL
1 campanha × 1 dia
o agregado do outro sistema

Grain. A primeira decisão de modelação — e aquela de que todas as outras dependem.

Três tabelas, **três grains diferentes**. Não é um defeito do modelo: é o mundo a chegar assim.

Fact ou dimension?

DIMENSION — DESCREVE

Quem, o quê, onde

- Poucas linhas (2, 6, 8, 7)
- Mudam devagar — SCD tipo 1
- Atributos descritivos

`dim_workspace` · `dim_traffic_source`
`dim_campaign` · `dim_offer`

FACT — REGISTA

O que aconteceu

- Muitas linhas, só crescem
- Um facto passado não muda
- Chaves + métricas + contexto

`fact_conversions` · `fact_campaign_costs`
`fact_daily_control`

Teste rápido: se a frase natural é *«é uma campanha»*, é dimension. Se é *«aconteceu às 14:32»* ou *«entre as 14h e as 15h»*, é fact.

O nosso modelo



ARMADILHA · FAN-OUT

Juntar `fact_conversions` com `fact_campaign_costs` e somar a seguir repete o `cost` de cada hora **uma vez por cada conversão**. Agrega-se cada fact ao seu grain e só depois se juntam os *resultados*.

O data catalog

Tabela	1 linha =	Chave primária	Linhas/dia
dim_workspace	1 workspace	`workspace_id`	2 · estática
dim_traffic_source	1 fonte de tráfego	`traffic_source_id`	6 · estática
dim_campaign	1 campanha	`campaign_id`	8 · estática
dim_offer	1 oferta	`offer_id`	7 · estática
fact_conversions	1 conversão	`click_id` + `conversion_type_id` + `postback_timestamp`	≈ 170
fact_campaign_costs	1 campanha × 1 hora × 1 provider	`datetime` + `campaign_id` + `investment_provider`	≈ 190
fact_daily_control	1 campanha × 1 dia	`date` + `campaign_id`	6

E 4 VIEWS POR CIMA

`v_campaign_daily` · `v_traffic_source_daily` · `v_offer_daily` · `v_reconciliation_daily`

PORQUE É QUE ISTO EXISTE

Uma tabela sem grain, sem chave e sem dono não é um ativo — é um ficheiro. O catálogo é o sítio onde isso está escrito.

Uma entrada do catálogo

`fact_campaign_costs` — 1 linha = 1 campanha × 1 hora × 1 *investment provider*

Coluna	Tipo	Nulo?	De onde vem
datetime	TIMESTAMPTZ	não	início da hora, UTC · **PK** · janela de extração
cost_date	DATE	não	*calculada* no transform · a data dos relatórios
campaign_id	TEXT	não	FK → `dim_campaign` · **PK**
investment_provider	TEXT	não	enum de 16 valores · **PK**
cost	NUMERIC(14,4)	não	USD · métrica aditiva · `CHECK >= 0`
clicks	BIGINT	**sim**	`NULL` = o provider não reporta
impressions	BIGINT	**sim**	`NULL` = o provider não reporta
loaded_at / updated_at	TIMESTAMPTZ	não	1.ª carga / última recarga

Nome, tipo, se pode ser nulo, de onde vem e o que lhe fizemos. **Nada disto se descobre olhando para o `CREATE TABLE`.**

Escolher a chave é escolher o grain

```
PRIMARY KEY (click_id,
             conversion_type_id,
             postback_timestamp)
```

Não há `conversion_id` na origem. A chave é **composta e natural**.

Um mesmo click pode converter **mais do que uma vez**: tipo 1 (o depósito), tipo 2 (o upsell), em momentos diferentes.

Por isso `click_id` sozinho **não é chave**.

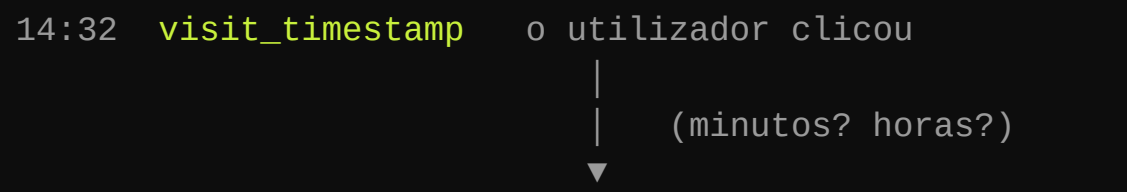
Tabela	Chave primária
fact_conversions	`click_id` + `conversion_type_id` + `postback_timestamp`
fact_campaign_costs	`datetime` + `campaign_id` + `investment_provider`
fact_daily_control	`date` + `campaign_id`

Cada chave **é** o grain, escrito em SQL. Se a chave admitir duas linhas para o mesmo acontecimento, o grain está errado.

E é esta chave que vai tornar a carga **idempotente** — às 16:00.

O DOMÍNIO E O MODELO

Duas datas, dois usos



EXTRAI-SE POR POSTBACK

É quando o dado **nos chega**. A janela da run tem de usar esta, senão uma conversão reportada hoje sobre uma visita de ontem nunca seria apanhada.

REPORTA-SE POR VISITA

`visit_date` é o dia UTC da visita. É a data dos relatórios: põe a receita no **mesmo dia do custo** que a gerou.

O custo tem o mesmo desenho: extrai-se por `datetime` (a hora), reporta-se por `cost_date`.

Métricas: base ou derivada?

BASE — GUARDAM-SE

`cost` · `clicks` · `impressions` · `revenue`

Aditivas: somam-se por qualquer dimensão e continuam a fazer sentido. `conversions` é `count(*)`.

DERIVADAS — CALCULAM-SE

CPC	`cost / clicks`	CR	`conv / clicks`
CPM	`cost / impr × 1000`	CPA	`cost / conv`
CTR	`clicks / impr`	EPC	`revenue / clicks`
ROAS	`revenue / cost`	ROI	`profit / cost`

Não guardar o que podemos calcular corretamente depois.

Soma primeiro. Divide depois.

dia	revenue	cost	roas
1 Out	60	60	1,0
2 Out	100	20	5,0

ERRADO

`AVG(roas)` = (1,0 + 5,0) / 2 = **3,0**

CERTO

`SUM(revenue) / SUM(cost)` = 160 / 80 = **2,0**

Um rácio não é aditivo.

Gastámos 80, rendeu 160. O ROAS dos dois dias é **2,0** — não 3,0.

A média dos rácios dá 50 % a mais, e o erro cresce quanto mais desiguais forem os dias.

Por isso as views agregam as métricas *aditivas* e só calculam o rácio no fim. E qualquer dashboard por cima delas tem de fazer o mesmo.

`NULLIF(x, 0)` em todas as divisões: um dia sem conversões tem CPA **indefinido**, não um erro.

Nulo não é zero

provider	cost	clicks	impressions
exoclick	12,40	380	52 000
trafficstars	8,10	210	31 000
pushground	5,00	160	**NULL**

`pushground` e `reactiveclicks` **não reportam** impressions. Não é zero: é *não sei*.

ERRADO

CTR = 750 / 83 000 = **0,90 %**

Usa clicks que não têm impressões correspondentes. Inflaciona.

CERTO

CTR = 590 / 83 000 = **0,71 %**

Só os clicks **das linhas que reportam impressions**.

Na view: `sum(clicks) FILTER (WHERE impressions IS NOT NULL)`. O mesmo para o CPM, com o custo.

API E DATA CONTRACT · DEMO 1

A nossa fonte

Endpoint	Janela sobre
/health	—
/workspaces	dimensão
/traffic-sources	dimensão
/campaigns	dimensão
/offers	dimensão
/conversions	`postback_timestamp`
/campaign-costs	`datetime`
/daily-control	`date`

```
{ "data":  [ ... ],
  "count": 100,
  "total": 437,
  "limit": 100,
  "offset": 0 }
```

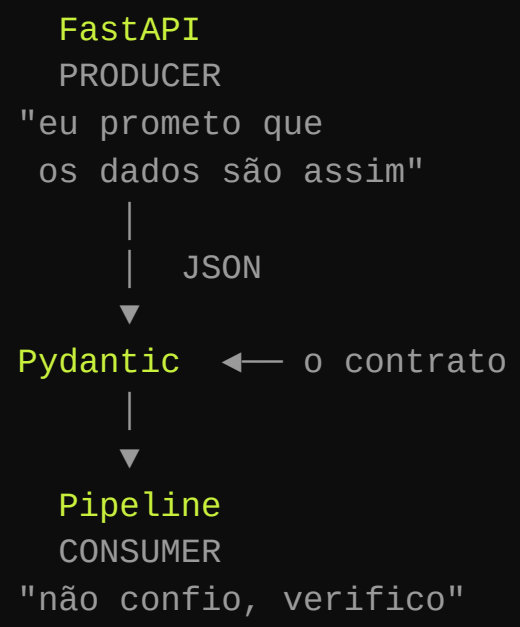
Pedi 100. Existem 437.
Como é que a pipeline sabe?

Sem `total`, uma extração parcial parece uma extração completa. É o pior tipo de bug em dados: não dá erro.

≈170 conversões · ≈190 linhas de custo · 6 de daily control, por dia.

API E DATA CONTRACT

Producer → Consumer



Uma API responder 200 não significa que os dados estejam corretos.

O producer valida o que **emite** — e só apanha os bugs dele próprio, se a validação existir.

O consumer valida o que **recebe** — e apanha qualquer desvio, venha de onde vier.

Por isso a pipeline valida outra vez, mesmo sabendo que a API já validou.

O contrato, em código

```
class CampaignCost(Contract):                    # extra="forbid"
    datetime:            AwareDatetime           # sem timezone -> erro
    campaign_id:         EntityId                # UUID estrito: "123" nao passa
    investment_provider: InvestmentProvider      # enum: 16 valores
    cost:                Money                   # >= 0, sem NaN/Infinity
    clicks:              int | None = None       # OPCIONAL: None != 0
    impressions:         int | None = None

    @field_validator("datetime")
    def must_be_top_of_the_hour(cls, v):         # regra do negocio, no contrato
        if v.minute or v.second:
            raise ValueError("deve ser o inicio de uma hora")
        return v
```

tipos estritos | enums fechados | constraints | opcionais explícitos | regras do negócio | campos desconhecidos = erro

API E DATA CONTRACT

Schema drift

ANTES

```
{ "revenue": 29.90,
  "cost": 12.40 }
```

DEPOIS

```
{ "revenue_amount": 29.90,
  "cost_amount": 12.40 }
```

HTTP 200 | 120 ms | serviço UP | JSON válido

Está tudo bem? NÃO.

Pydantic: `revenue → missing` e `revenue_amount → extra_forbidden`, em **172 de 172** registos. É a única coisa no sistema inteiro que dá por isto.

Extract

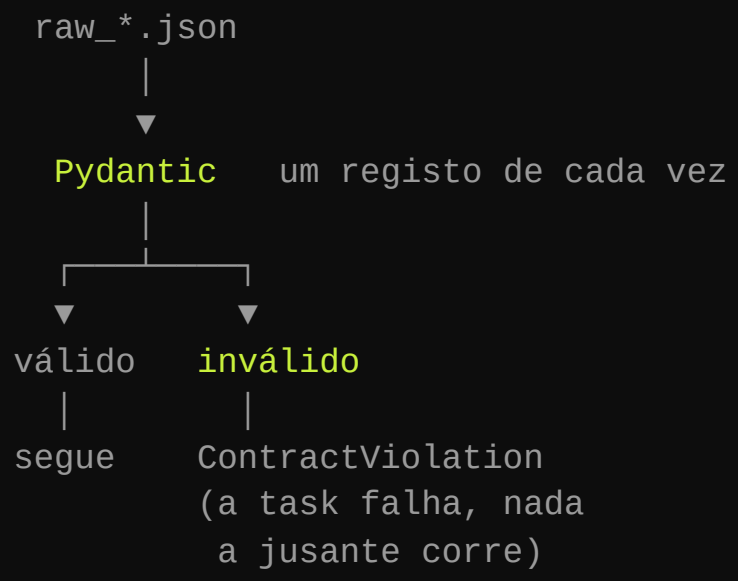
```
API  ──►  raw_conversions.json
          raw_campaign_costs.json
          raw_daily_control.json
          raw_campaigns.json  ...

data/staging/20261003T000000/
     ▲
     a pasta é a janela,
     não o relógio
```

```python
while True:
    page = get(path, limit=500, offset=offset)
    rows += page["data"]
    offset += len(page["data"])
    if offset >= page["total"]:
        break
if len(rows) != total:
    raise ExtractError(...)
```

O **XCom** leva caminhos e contagens — nunca os dados. Vive na base de metadados do Airflow e é para mensagens pequenas.

Validate




```
conversions   13 de 172 INVÁLIDOS
  campo=revenue       greater_than_equal
  campo=country_code  pattern_mismatch

conversions   172 de 172 INVÁLIDOS
  campo=revenue         missing
  campo=revenue_amount  extra_forbidden
```

Valida **todos** os registos, não pára no primeiro. `13 em 172` é lixo pontual; `172 em 172` é o schema que mudou.

Transform

Antes	Depois	Porquê
`"pt"` / `" PT "`	`"PT"`	espaços e caixa partem os `GROUP BY`
`2026-10-03T10:00+01:00`	`2026-10-03T09:00Z`	um único fuso, sempre
`visit_timestamp`	`+ visit_date`	a data de negócio, calculada uma vez
`datetime` (hora)	`+ cost_date`	idem, do lado do custo
`clicks: NaN`	`clicks: NULL`	«não sei» continua a ser «não sei»

Pandas. Funções puras: `DataFrame → DataFrame`.

Aqui **não** se calcula CPA nem ROAS. Métricas derivadas não se guardam ao nível do facto.

Load

1 · dimensões	dim_workspace	dim_traffic_source	dim_campaign	dim_offer
2 · factos	fact_conversions	fact_campaign_costs	fact_daily_control	

DIMENSÕES PRIMEIRO

As FKs dos factos apontam para elas. Carregar uma conversão de uma campanha que ainda não existe é um erro de integridade referencial — e ainda bem.

E DENTRO DAS DIMENSÕES, ORDEM TAMBÉM

`dim_campaign` referencia `dim_workspace` e `dim_traffic_source`. A ordem não é estética: é o que a base de dados exige.

UMA TRANSAÇÃO POR PASSO

Ou entram todas as tabelas do passo, ou nenhuma. Falhar a meio não deixa meia carga.

APACHE AIRFLOW

Então porque precisamos de Airflow?

```
$ python pipeline.py
  172 conversoes, 190 linhas de custo, 6 de daily control  -  18 s
```

Quem o executa amanhã às 4h?

E se falhar a meio?

E se eu precisar de repetir ontem?

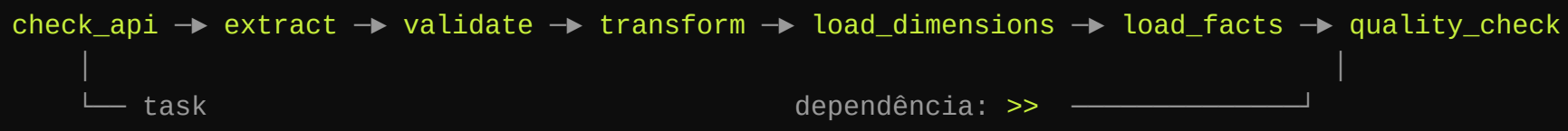
E se uma etapa depender da anterior?

Onde vejo o que aconteceu na semana passada?

Nenhuma destas perguntas é sobre dados.
São todas sobre operação.

APACHE AIRFLOW · DEMO 2

Anatomia do nosso DAG



SCHEDULE
`@daily`

CATCHUP
`False`

MAX_ACTIVE_RUNS
`1`

RETRIES
`2`

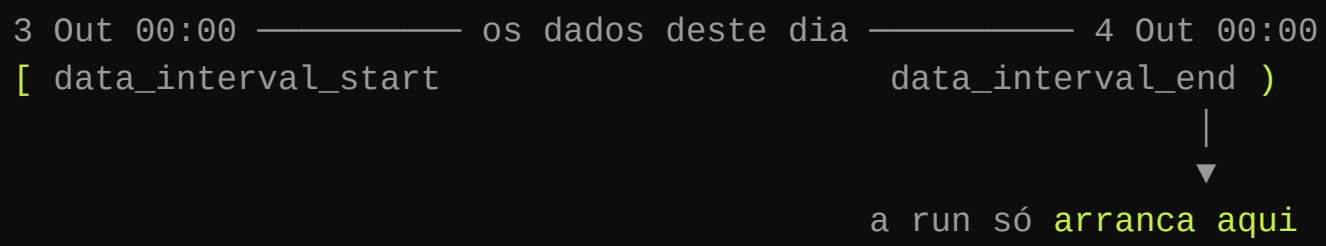
RETRY_DELAY
`30 s`

EXECUTION_TIMEOUT
`2 min`

Airflow não é a ETL.
Airflow orquestra a ETL.

APACHE AIRFLOW

Data interval



Uma run representa uma **fatia de dados**, não a hora a que executou.

O QUE A RUN FAZ COM ISTO

Passa a janela ao `extract`, que a envia a cada endpoint como `start_date` / `end_date`. Fim inclusivo nunca: `[início, fim)`.

PORQUE É QUE ISTO IMPORTA

A run não processa «o que houver agora». Processa **um dia concreto** — e esse dia pode ser reprocessado a qualquer momento com o mesmo resultado.

APACHE AIRFLOW

Catchup vs backfill

CATCHUP — O SCHEDULER DECIDE

```
start_date      hoje
    |             |
    ├─ 1 Out ─────┤   runs criadas
    ├─ 2 Out      |   automaticamente
    ├─ 3 Out      |   ao ativar o DAG
    └─ ...        |
```

Com `catchup=True` e um `start_date` distante, ativar o DAG dispara centenas de runs de uma vez.

Aqui: `catchup=False`.

BACKFILL — EU DECIDO

```
airflow backfill create \
  --dag-id tracking_pipeline \
  --from-date 2026-10-01 \
  --to-date   2026-10-05
→ 5 runs, uma por dia
```

Explícito, com limites que eu escolhi, quando eu quiser.

Só é seguro porque a carga é idempotente. Se algum desses dias já estava carregado, não se duplica nada — voltamos a isto às 16:00.

FAILURES

Agora vamos partir a pipeline

1
HTTP 500
a API devolve erro

2
API lenta
o pedido não volta

3
HTTP 429
rate limit, e recupera

4
Schema drift
dois campos mudaram de nome

5
Payload inválido
`revenue` e `clicks` negativos

6
Run repetida
o mesmo dia outra vez

7
Dois sistemas discordam
o daily control não bate

Um endpoint `/admin/mode` muda o comportamento da API. **Só para a aula** — em produção isto seria chaos engineering, isolado e autenticado.

FAILURES

Retry não é error handling

Retry é para erros transitórios.

RETRY FAZ SENTIDO

- `500` — o servidor pode recuperar
- `timeout` — a rede pode melhorar
- `429` — a quota renova-se
- deadlock na base de dados

O erro **pode** desaparecer sozinho.

RETRY SÓ ADIA A FALHA

- schema drift
- `revenue` negativo
- bug na query
- credenciais erradas

O erro é **determinístico**: daqui a 30 s está igual.

Por isso `validate` e `quality_check` têm `retries=0`. Falham logo, e quem lê o log vê o erro mais cedo.

FAILURES · DEMO 3 · DEMO 4

500 e 429: quando o retry salva

HTTP_500 — O RETRY NÃO SALVA

```
check_api  tentativa 1   500
           ...30 s...
           tentativa 2   500
           ...30 s...
           tentativa 3   500
           ──────────────────
           failed
extract .......... upstream_failed
validate ......... upstream_failed
load_* ........... upstream_failed
```

Nada escrito na base de dados. A falha parou tudo.

RATE_LIMIT — O RETRY SALVA

```
check_api  tentativa 1   429
           ...30 s...
           tentativa 2   429
           ...30 s...
           tentativa 3   200
           ──────────────────
           success
o DAG continua, e termina em verde
```

Uma hora depois ninguém sabe que isto aconteceu.

No segundo caso, o retry resolveu o problema — ou escondeu-o?

FAILURES · DEMO 5

A API está lenta

CHECK_API
`GET /health`
200 em 8 ms

EXTRACT
`GET /conversions`
timeout aos 5 s

A API está saudável?

Um health check não prova que o serviço consegue servir os dados.

TIMEOUT 1 · HTTP
`5 s` — o pedido desiste. Apanha a API pendurada.

TIMEOUT 2 · TASK
`2 min` — o Airflow mata a tentativa. Apanha tudo o resto.

FAILURES · DEMO 6

Schema drift, ao vivo

HTTP 200 | latência 90 ms | serviço UP | JSON válido | Pydantic falha

Availability ≠ correctness.

O QUE A INFRAESTRUTURA VÊ

Nada. Uptime 100 %, latência normal, zero erros 5xx. Qualquer painel de monitorização clássico fica verde.

O QUE A PIPELINE VÊ

`validate` falha: `172 de 172` conversões e `190 de 190` custos inválidos. Tudo a jusante fica `upstream_failed`.

IDEMPOTÊNCIA E QUALIDADE

O Airflow vai executar de novo

retry automático | clear task na UI | trigger manual | backfill | conversões tardias

todos levam ao mesmo → o mesmo período, processado outra vez

Vamos duplicar os dados?

Não é uma hipótese remota. Esta tarde já disparámos o dia 3 de outubro **cinco vezes**.

IDEMPOTÊNCIA E QUALIDADE · DEMO 7

Idempotency

Executar uma vez ou dez vezes deixa o sistema no mesmo estado.

execução	INSERT simples	upsert
run 1	172 linhas	172 linhas
run 2	344	172
run 3	516	172

172 conversões distintas nos dois casos. À esquerda, a receita do dia passou de **4 531,80** a **13 595,40 USD**.

```
INSERT INTO fact_conversions (...)
VALUES (...)
ON CONFLICT (click_id,
             conversion_type_id,
             postback_timestamp)
DO UPDATE SET revenue = EXCLUDED.revenue,
              updated_at = now();
```

Três peças: a **chave composta** identifica a linha, o **upsert** diz o que fazer quando já existe, e a origem é **determinística**.

IDEMPOTÊNCIA E QUALIDADE

Validate ≠ Data Quality

VALIDATE — ANTES DO LOAD

Este registo tem a forma certa?

- Pydantic, uma linha de cada vez
- schema, campos obrigatórios e opcionais
- tipos, enums, UUIDs
- `cost >= 0`, país com 2 letras, hora certa

QUALITY_CHECK — DEPOIS DO LOAD

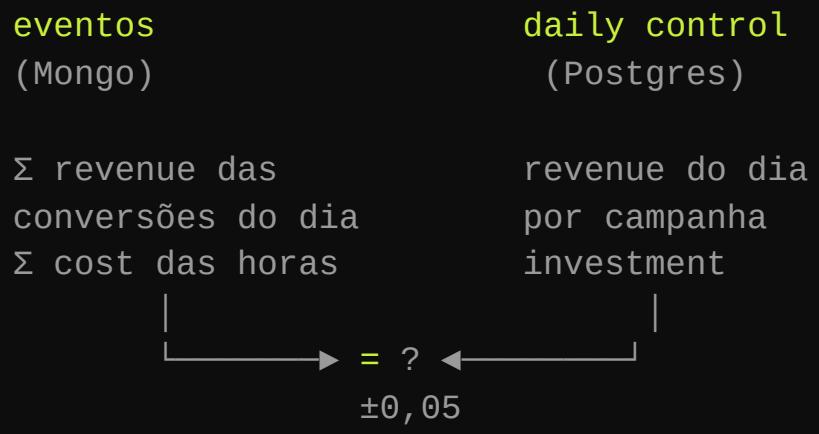
Este conjunto faz sentido?

- SQL, sobre a janela já carregada
- `row count > 0` · freshness (< 6 h)
- chaves compostas únicas · FKs válidas
- `postback >= visita` · fonte coerente
- **reconciliação com o daily control**

Corre **todas** as verificações, mesmo depois da primeira falhar, e falha no fim com a lista completa. Um relatório, não um alarme isolado.

IDEMPOTÊNCIA E QUALIDADE · DEMO 8

Reconciliação: dois sistemas, um número



Nenhuma *constraint* apanha dois sistemas a discordar.

É o único check que compara a nossa pipeline com **uma fonte independente**. Tudo o resto verifica se somos coerentes connosco próprios.

EXCETO QUANDO É À MÃO

`is_manual = true` significa que uma pessoa escreveu aquele número. Nesse caso **não** se exige que bata certo — exigir seria transformar uma decisão humana num alarme diário.

PIPELINE

SUCCESS

sete tasks verdes, 18 segundos, zero retries

DATA

ERRADO

metade das linhas, receita a zero, ontem em vez de hoje

Uma pipeline verde pode entregar dados errados.

O job ter corrido com sucesso não diz **nada** sobre os dados.

OBSERVABILIDADE

Como sabemos que está tudo bem sem ficar a olhar para o Airflow?

PIPELINE
O job correu?
sucesso e falha de DAG e tasks, retries, duração, atraso do scheduler

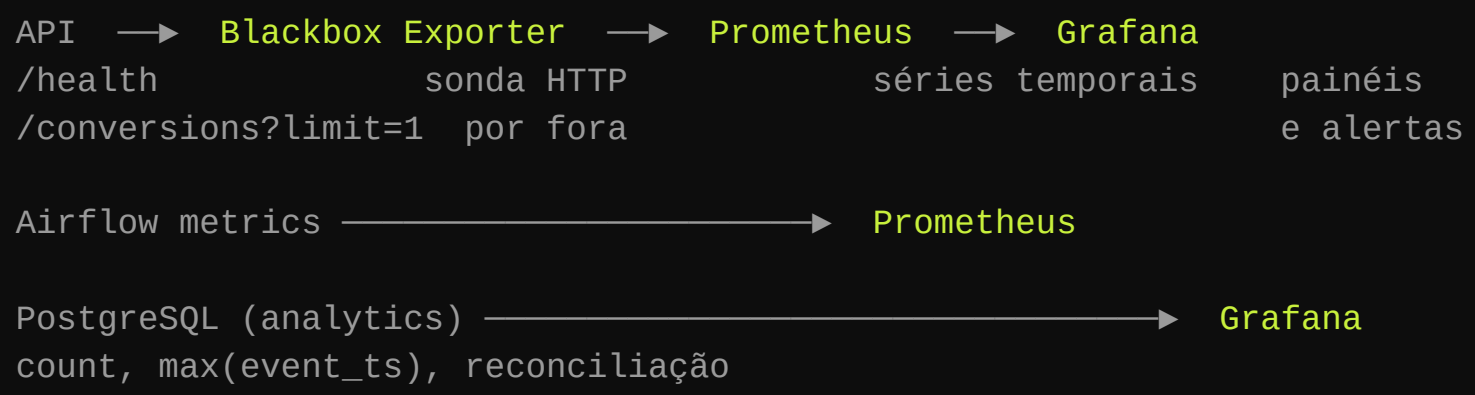
SOURCE
A fonte está bem?
disponibilidade, latência, códigos HTTP do producer

DATA
Os dados estão certos?
volume, freshness, duplicados, falhas de validação, reconciliação

Dos sete cenários de hoje, **três** — schema drift, payload inválido e daily control a discordar — passam despercebidos a qualquer monitorização de disponibilidade.

OBSERVABILIDADE

Próxima evolução



Observability começa por saber **que pergunta queremos responder**.

Não está instalado, de propósito. Está escrito em `docs/observability-roadmap.md` — três serviços novos no `docker-compose.yml` e uma pasta de configuração.

FECHO

O sistema completo

```
API          ← producer
  ↓
Airflow      ← orquestra
  ↓
Contract     ← Pydantic
  ↓
Pandas       ← transforma
  ↓
PostgreSQL   ← upsert, chave composta
  ↓
Analytics    ← soma, depois divide
```

retries · idempotency · quality · reconciliação · observability

TAKEAWAYS

- **Grain antes de código.**
- **Escolher a chave é escolher o grain.**
- **Airflow orquestra; não transforma.**
- **Retry é para falhas transitórias.**
- **Soma primeiro, divide depois. E nulo não é zero.**
- **Reprocessar exige idempotência.**

FECHO · DISCUSSÃO

Se esta pipeline estivesse em produção amanhã, o que ainda faltaria?

Sem respostas no slide. A sala tem 8 minutos.

ANEXO · REFERÊNCIA

O catálogo ao detalhe

As cinco páginas seguintes não se leem em aula. São para projetar quando alguém perguntar «e esta coluna, o que é?», e para ler depois.

Versão completa, com linhagem e glossário: `docs/data-catalog.md`.

ANEXO · REFERÊNCIA

As quatro dimensões

Dimensão	Colunas	Nota
dim_workspace	`workspace_id` **PK** · `name`	2 linhas
dim_traffic_source	`traffic_source_id` **PK** · `name` · `currency_code` · `deleted`	*soft delete*: não se apaga, as facts antigas apontam para cá
dim_campaign	`campaign_id` **PK** · `name` · `geo` · `traffic_source_id` FK · `workspace_id` FK · `is_paused` · `created`	`geo` é ISO-2 ou `WW` (worldwide)
dim_offer	`offer_id` **PK** · `name` · `country_code` ? · `affiliate_network_id` · `affiliate_network_name` · `payout` · `currency_code` · `deleted`	`country_code` nulo = oferta global

EM TODAS

`loaded_at` e `updated_at` : quando a linha apareceu e quando foi recarregada.

SCD TIPO 1

Mudar o nome de uma campanha substitui o antigo. Perde-se o histórico — é uma escolha, não um esquecimento.

A REDE DE AFILIAÇÃO

Vive como **atributos da oferta**, sem dimensão própria. Passa a dimensão quando tiver atributos seus.

ANEXO · REFERÊNCIA

fact_conversions

1 linha = 1 conversão (um postback) · ≈ 170 por dia

Coluna	Tipo	Nulo?	De onde vem
click_id	TEXT	não	32 hex, da origem · **PK**
conversion_type_id	INTEGER	não	1 = principal, 2 = secundária · **PK**
postback_timestamp	TIMESTAMPTZ	não	quando a rede nos reportou · **PK** · janela de extração
visit_timestamp	TIMESTAMPTZ	não	quando o utilizador clicou
visit_date	DATE	não	*calculada* no transform · a data dos relatórios
campaign_id	TEXT	não	FK → `dim_campaign`
traffic_source_id	TEXT	não	FK → `dim_traffic_source` · tem de bater com a da campanha
offer_id	TEXT	não	FK → `dim_offer`
country_code	CHAR(2)	não	país do visitante, ISO-2
revenue	NUMERIC(14,4)	não	USD · métrica aditiva · `CHECK >= 0`
revenue_original_currency	NUMERIC(14,4)	não	o mesmo, na moeda da oferta

ANEXO · REFERÊNCIA

fact_daily_control e as views

1 linha = 1 campanha × 1 dia · 6 por dia

Coluna	Tipo	De onde vem
date	DATE	o dia · **PK**
campaign_id	TEXT	FK · **PK**
revenue	NUMERIC(12,2)	receita do dia, backoffice
investment	NUMERIC(12,2)	investimento do dia
is_manual	BOOLEAN	alguém escreveu à mão

Duas casas decimais, não quatro: por isso a reconciliação tem tolerância de 0,05.

View	1 linha =
v_campaign_daily	1 campanha × 1 dia · todas as métricas
v_traffic_source_daily	1 fonte × 1 dia · a fonte vem do click
v_offer_daily	1 oferta × 1 dia · **sem custo**
v_reconciliation_daily	1 campanha × 1 dia · eventos vs daily control

`v_offer_daily` não tem CPA nem ROAS: o tráfego compra-se para uma *campanha*, não para uma oferta. A ausência é informação.

ANEXO · REFERÊNCIA

Glossário de métricas

ADITIVAS — GUARDADAS

cost	`fact_campaign_costs`
clicks	`fact_campaign_costs`
impressions	`fact_campaign_costs`
revenue	`fact_conversions`
conversions	`count(*)`

DERIVADAS — CALCULADAS NAS VIEWS

CPC	`cost / clicks`	CR	`conv / clicks`
CPM	`cost / impr × 1000`	CPA	`cost / conv`
CTR	`clicks / impr`	EPC	`revenue / clicks`
ROAS	`revenue / cost`	profit	`revenue − cost`
ROI	`profit / cost`		

soma primeiro, divide depois

NULLIF em todas as divisões

nulo não é zero

cada fact ao seu grain (fan-out)

custo no dia da hora, conversão no dia da visita

ANEXO · REFERÊNCIA

Linhagem e regras por camada

```
API /conversions     ─▶ raw_conversions.json     ─▶ validate ─▶ transform ─▶ transformed_*.csv ─▶ fact_conversions
API /campaign-costs  ─▶ raw_campaign_costs.json  ─▶ validate ─▶ transform ─▶ transformed_*.csv ─▶ fact_campaign_costs
API /daily-control   ─▶ raw_daily_control.json   ─▶ validate ─▶ transform ─▶ transformed_*.csv ─▶ fact_daily_control
API /campaigns ...   ─▶ (idem)                                                                 ─▶ dim_*

                        fact_* + dim_*  ─▶ v_campaign_daily · v_traffic_source_daily · v_offer_daily
                        fact_* + fact_daily_control  ─▶ v_reconciliation_daily ─▶ quality_check
```

CAMADA 1 · CONTRATO
Pydantic, na entrada: schema, tipos, enums, constraints de campo, hora certa.

CAMADA 2 · BASE DE DADOS
Constraints: PKs compostas, FKs, `CHECK >= 0`, `NOT NULL`.

CAMADA 3 · CONJUNTO
SQL, depois do load: contagem, freshness, duplicados, FKs, coerência, reconciliação.